

ebb-ai · Changelog

Vitalii Borovyk

Changelog

All notable changes to ebb-ai will be documented in this file.

The format is based on [Keep a Changelog](#), and this project adheres to [Semantic Versioning](#).

Unreleased

0.5.0 — 2026-05-13

Theme: “Control and reliability.” Answers the four product questions raised in review: how to make sure the user gives a clear request, what form the result takes, what to do if the user changes their mind, and how to mitigate the operational failure modes (rate-limits, concurrent ticks, secrets in receipts).

Added — control surface

- **cancel_task (MCP) + Scheduler.cancelTask (TS) + Scheduler.cancel_task (Python).** Idempotent cancellation. If the task is queued/scheduled, status transitions to cancelled. Already-terminal tasks return unchanged. Throws only on unknown task id.
- **expedite_task (MCP) + Scheduler.expediteTask + Scheduler.expedite_task (Python).** Dispatch a queued provider-call task immediately, bypassing its carbon window. Receipt records intensitySource: "expedited" to preserve the audit trail's honesty.
- **update_deadline (MCP) + Scheduler.updateDeadline + Scheduler.update_deadline (Python).** Re-score and reschedule a queued task against a new deadline. Throws if the task is already running or in a terminal state.
- **retry_task (MCP) + Scheduler.retryTask + Scheduler.retry_task (Python).** Re-dispatch a failed task. Receipt overwrites the previous.

Added — task-clarity layer

- **schedule_task dry_run: true (MCP) + Scheduler.previewProviderCall (TS).** Returns the planned dispatch — chosen window, estimated carbon,

band, batch eligibility — **without** persisting anything. Useful for confirmation flows before the agent commits to schedule a task.

- **Prompt validation.** `schedule_task` rejects empty / whitespace prompts at the boundary instead of dispatching garbage hours later.

Added — result delivery

- **output_path field on ProviderCallSpec.** When set, the dispatcher writes { `taskId`, `result`, `receipt` } as JSON to that path on success. Useful for inbox patterns and file-watchers.
- **prompt on CarbonReceipt.** The receipt now retains the prompt (redacted by default — see below) so the audit ledger is complete.
- **totalTokens on CarbonReceipt.** Total token count reported by the provider, when available.

Added — failure-mode mitigation

- **Retry with exponential backoff.** Provider-adapter dispatches retry 3 times at 1 s / 4 s / 16 s on transient errors (429 rate-limit, 5xx, network errors). Non-retryable errors (4xx other than 429) fail fast.
- **Row-level claim in tick.** Two concurrent `ebb tick` processes pointing at the same SQLite file no longer race-dispatch the same task — only the process whose `UPDATE tasks SET status='running' WHERE task_id=? AND status='scheduled'` changed exactly one row owns the dispatch. Concurrency test verified deterministic at 10/10 reruns.
- **Receipt redaction.** `redactInReceipt` field on `ProviderCallSpec` controls regex redaction of secrets in the stored prompt. Omitted = default patterns (API-key shapes, bearer tokens). [] = no redaction. The dispatched call uses the original prompt; only the receipt is redacted.

Added — Linux daemonization (was templates-only in v0.4)

- **Real systemd .service + .timer generation** in `@ebb-ai/cli` (platform/`linux.ts`). `ebb install --laptop` writes `~/.config/systemd/user/ebb-tick.service` and `ebb-tick.timer` plus the laptop-wake helper script; emits the exact `systemctl --user daemon-reload && systemctl --user enable --now ebb-tick.timer next-step` line.
- **rtcwake wake-event support** on Linux mirroring the macOS `pmset schedule wake path`. `ebb register-wake <task-id>` is now cross-platform.

Added — Python parity

- **Scheduler.tick(adapters)** Python — drains due provider-call tasks via the matching adapter, identical semantics to TS.
- **Scheduler.cancel_task / expedite_task / update_deadline / retry_task** Python.
- **ProviderCallSpec, TickResult, TickResultEntry** dataclasses in `ebb_ai.types`.
- **Row-level claim** in the Python `_TaskStore` matches TS.
- **body_json column migration** in the Python SQLite store matches TS.

Changed

- @ebb-ai/core 0.4.0 → **0.5.0**.
- @ebb-ai/mcp 0.4.0 → **0.5.0**.
- @ebb-ai/cli 0.4.0 → **0.5.0**.
- ebb-ai (Python) 0.3.0 → **0.5.0** (skips 0.4 to align).
- MCP server now exposes **8 tools** (was 4): the four v0.3 tools plus `cancel_task`, `expedite_task`, `update_deadline`, `retry_task`.
- TaskStatus union gains "cancelled". IntensitySource gains "expedited".

Tests

- TS core: **65 passed** (was 48; +17 for v0.5 — preview, redact, output_path, retry/backoff, cancel/expedite/update/retry, row-level claim).
- MCP server: **8 passed** (unchanged at the protocol-stub level; the new tools' wire shape is covered by the underlying scheduler tests).
- @ebb-ai/cli: **21 passed** (was 13; +8 for Linux platform).
- Python: **75 passed** (was 56; +19 for v0.5 tick / cancel / expedite / update / retry / concurrency).
- **Total across the project: 169 passing.**

Known limitations (planned for v0.6)

- Single-writer SQLite holds. WAL multi-writer is queued.
- Lenient deadline parser ("tomorrow 8am") not yet shipped; only ISO-8601 datetimes with timezone offset are accepted.
- Stuck-running detection is documented but not yet auto-recoverable; tasks stuck in running > 1h are reported but not auto-failed.
- Webhook delivery (output: { webhook: "..."}) deferred — file output and pull-by-id remain the v0.5 delivery modes.
- Python pyebb CLI deferred — operators use the TS ebb CLI.

0.4.0 — 2026-05-12

Added

- **@ebb-ai/cli** — new package shipping the ebb binary. Five subcommands: `tick`, `install`, `queue list`, `receipts list`, `register-wake`. Cross-platform shell entrypoint for the v0.4 always-on story.
- **ebb tick** — one-shot or `--daemon` mode. Opens the SQLite ledger, finds tasks whose `scheduled_for` has elapsed, dispatches via Anthropic/OpenAI adapters, writes the receipt. This closes the v0.3 gap where deferred tasks died with the MCP host process.
- **ebb install --laptop | --server** — macOS launchd plist generator. `--laptop` additionally drops a `laptop-wake.sh` helper that registers `pmset schedule wake` events 30s before each scheduled task. `--server` skips the wake step.
- **ebb register-wake <task-id>** — schedules a macOS wake event via `pmset` for a specific task. `--dry-run` prints the exact command without running it.

Sudoers entry required for non-root invocation; the command shape is printed every time so it can be pre-authorized.

- **@ebb-ai/core v0.4:**
 - New `Scheduler.enqueueProviderCall(spec, opts)` method — persists a JSON-serializable provider-call task body. Closures still work via `defer()` but are still in-process-only; provider-calls survive restart.
 - New `Scheduler.tick(adapters)` method — drains due tasks persistently. Returns `{ inspected, dispatched, failed }`.
 - New types `ProviderCallSpec`, `TickResult`, `TickResultEntry`.
 - SQLite schema migration: added `body_json` column via idempotent `ALTER TABLE` so v0.3 DB files are picked up without manual intervention.
- **Platform abstractions** — `packages/cli/src/platform/macos.ts` ships `caffeinateWhilePending`, `pmsetScheduleWake`, `pmsetCommand`, `formatPmsetDate`, `launchdPlist`. `linux.ts` and `windows.ts` ship template generators (`systemd / schtasks`) as stubs; full Linux + Windows runtimes land in v0.5.
- **Five new example folders** — `examples/cline/`, `examples/continue/`, `examples/zed/`, `examples/windsurf/`, `examples/pi/`. Each ships a README + the host's native MCP-config shape (`mcp_settings.json / config.yaml / native context_servers block / mcp_config.json / AGENTS.md` skill stanza). Brings supported MCP-host count from 5 → 10.

Fixed

- TypeScript SQLite store crashed at runtime with `ERR_AMBIGUOUS_MODULE_SYNTAX` because the `better-sqlite3` loader used CJS-style `require()` inside an ESM emit. Replaced with `import { createRequire } from "node:module"; const requireFn = createRequire(import.meta.url);`. Vitest tests passed (vite polyfills `require`); the production `dist/` binary did not until this fix.
- Wrong dashboard pnpm filter in `README.md` and `QUICKSTART.md`: `pnpm --filter ebb-dashboard dev` corrected to `pnpm --filter @ebb-ai/dashboard dev`.

Changed

- `@ebb-ai/core` 0.3.0 → **0.4.0**.
- `@ebb-ai/mcp` 0.3.0 → **0.4.0**.
- `@ebb-ai/cli` initial release at **0.4.0**.
- Python `ebb_ai` stays at 0.3.0 in this drop; Python parity for the CLI and `tick()` method lands in v0.5 (see roadmap).

Tests

- TypeScript core: **48 passed** (was 41; +7 for the `tick / persisted provider-call` path).
- MCP server: **8 passed** (unchanged from v0.3).
- CLI: **13 passed** (new package: 8 platform-macos snapshots, 4 install dry-run / template, 1 tick env handling).
- Python: **56 passed** (unchanged in v0.4 per scope; v0.5 brings Python `tick` parity).

- Total across the project: **125 passing**.

Known limitations (planned for v0.5)

- Linux + Windows install paths only emit a template; no live daemonization. Use the printed systemd unit / schtasks command as a starting point.
- `pmset schedule wake` requires sudo per call. Users must add a sudoers entry for the ebb binary or run `ebb register-wake` manually.
- Closure-based `defer()` tasks are still in-process only. v0.4 intentionally only persists `provider_call` task bodies.
- Python port does not yet have a `tick()` method or a pyebb CLI.

0.3.0 — 2026-05-12

Added

- **recommend_window** — **planning endpoint**. New MCP tool + `recommendWindow()` (TS) / `recommend_window()` (Python) library functions. Returns the optimal execution time for a task **without** committing to schedule it. Output includes the chosen window's intensity, carbon estimate, percent savings vs. running now, batch eligibility, three alternative windows, and a one-line human-readable reasoning string. Pure read-only — does not touch the queue.
- **Cursor example**. `examples/cursor/` with `mcp.json` stanza and install README. Cursor ≥ 0.45 speaks MCP natively.
- **Claude Desktop example**. `examples/claude-desktop/` with config template and troubleshooting guide.
- **QUICKSTART.md** — four steps, five minutes. Linked from README.
- **Copy buttons** on every code block on the marketing site (`apps/site/copy-buttons.js`).
- **Launch hero** in README.md: a single-glance `recommend_window` example showing the canonical JSON output.

Changed

- `@ebb-ai/core 0.2.0` → **0.3.0**, `@ebb-ai/mcp 0.2.0` → **0.3.0**, `ebb-ai (Python) 0.2.0` → **0.3.0**.
- MCP server now exposes **4 tools** (was 3): `get_grid_forecast`, `recommend_window`, `schedule_task`, `check_queue_status`.
- Site `architecture.html` system diagram: text overflow fix on SCHEDULER CORE and MCP SERVER boxes; updated tool count.
- Site `index.html` “Why it matters” section: corrected Batch API framing. **Batch is not faster** — it trades latency (24h SLA) for the 50% discount. Carbon and dollars are the primary pitch; latency is a third-tier side effect. Same correction applied in README.md and `packages/core-py/README.md`.
- Inline `<code>` now wraps long file-paths via `word-break: break-all` (was overflowing `.int-card` content).

Tests

- TypeScript core: **41 passed** (was 26; +15 for recommend).

- MCP server: **8 passed** (was 6; +2 for recommend_window over the real MCP protocol).
- Python: **56 passed** (was 41; +15 for recommend).
- Total across the project: **105 passing**.

0.2.0 — 2026-05-12

Added

- **Provider adapters.** AnthropicAdapter and OpenAIAdapter in @ebb-ai/core/providers. Each adapter exposes dispatch() for sync calls and dispatchBatch() for the vendor Batch API (50% discount, 24h SLA). SDKs are peer dependencies and imported lazily, so missing-SDK callers still get a clear error rather than a module-load crash.
- **SQLite-backed durable queue.** Opt-in via new Scheduler({ dbPath }). Every state transition writes through to an on-disk audit ledger; records survive process restart and can be reloaded with Scheduler.listPersistedTasks() / Scheduler.loadPersistedTask().
- **Python port.** packages/core-py/ ships a complete Python 3.11+ package (ebb_ai) with asyncio scheduler, aiosqlite persistence from day one, and AnthropicAdapter + OpenAIAdapter mirrors. 41/41 pytest cases passing.
- **Dashboard MVP.** apps/dashboard/ is a Next.js 15 app router application with four pages (Home, Forecast, Plan, Queue), 6 grid regions, recharts visualizations, and live /api/grid/[region] + /api/queue endpoints. Mock fallback when no Electricity Maps key is set.
- **Site expansion.** apps/site/ now ships architecture.html (SVG diagram + 8-step data flow), roadmap.html (v0.1 → v1.0 with versioned status pills), and docs.html (categorized GitHub-linked index).
- Real MCP-protocol smoke test using InMemoryTransport.
- CHANGELOG.md, CONTRIBUTING.md, CODE_OF_CONDUCT.md, CODEOWNERS.
- docs/spec/01-priority-and-deadline-fields.md — draft proposal for upstream MCP spec engagement.

Changed

- @ebb-ai/core bumped to **0.2.0**, @ebb-ai/mcp bumped to **0.2.0**.
- Public surface expanded with AnthropicAdapter, OpenAIAdapter, TaskStore, and the supporting types BatchHandle, DispatchOptions, DispatchResult, ProviderAdapter, TaskStoreOptions.

Known limitations (planned for v0.3)

- Scheduler does not yet auto-route between sync and Batch paths based on deadline distance — adapters expose both, but the v0.2 scheduler still treats the task body as opaque. Wiring the routing decision into the scheduler is the lead v0.3 item.
- WattTime marginal-emissions feed is still pending.
- Multi-writer (multiple schedulers pointed at one DB) requires WAL mode; v0.2 SQLite store assumes single writer.

0.1.0 — 2026-05-12

Added

- @ebb-ai/core — TypeScript core library: Scheduler class, defer() function, pickBestWindow helper.
- @ebb-ai/mcp — Model Context Protocol server exposing get_grid_forecast, schedule_task, and check_queue_status tools over stdio.
- Mock and Electricity Maps grid feeds with automatic fallback when no API key is configured.
- Carbon budget enforcement: carbonBudgetG in DeferOptions is now honored; tasks fail with CarbonBudgetExceededError when no in-deadline window meets the budget.
- ISO-8601 deadline validation: invalid or past deadlines throw InvalidDeadlineError at the scheduler boundary and the MCP request validator.
- Reference integrations for Claude Desktop, Claude Code, OpenClaw, and OpenAI Codex CLI in examples/.
- Static landing site under apps/site/ (no JS, no framework).
- 24-week project plan in ROADMAP.md.

Fixed

- (Pre-v0.1.0 engineering review.) Carbon budget was previously captured but unenforced; schedule_task accepted unparseable and past deadlines silently; Electricity Maps fetch had no timeout; README and landing page used a non-existent region code "us-east"; Claude Code install flow documented a config path the client does not read. All addressed before v0.1 ship.

Known limitations (planned for v0.2)

- The MCP server schedules dispatch time but does not invoke the LLM itself. The agent calling the tool is expected to execute the prompt when the window arrives.
- Queue is in-memory; restarts lose state. SQLite-backed durable queue is planned for v0.2.
- No provider adapters yet for Anthropic Message Batches or OpenAI Batches APIs. Direct Batch routing lands in v0.2.
- Python port (ebb-ai PyPI) is a placeholder. v0.2.